E- Shop & Events – An Integrated E- commerce and Event Management Mobile Application

This project presents a modern, cross-platform mobile application that seamlessly combines online shopping and event booking into a single platform. Developed using Flutter and powered by Firebase services, the application supports role-based access for customers and sellers, enabling product browsing, cart management, order placement, event discovery, seat booking, and analytical sales. The system offers a user-friendly interface, real-time database integration, and efficient resource management, providing a unified digital solution for both e-commerce and event management needs.

Chapter 2: Objective

The primary objectives of this project are:

1. To create a cross-platform mobile application (using Flutter) that combines two essential services – e-commerce shopping and event management – into a single, user-friendly platform.
2. To provide a separate experiences for normal users (who can browse products, see recommended products, add to cart, place orders, book events, can see their order history, and can see and use the valid coupons for a discount) and sellers (who can add products/events, manage inventory, view customer orders, analyse product sales, and can make coupons for the user to get the discount).
3. For a secure user authentication using “Firebase Auth” and real-time data storage using “Firestore” with role-based access control.
4. And to integrate a professional image hosting service using “Cloudinary” for product and event images.
5. To enable cart persistence across sessions using ”SharedPreferences”, ensuring that a user’s cart is saved even after logging out and logging back in.
6. To allow cancellation of event bookings with automatic seat availability updates.
7. To equip sellers with an analytics screen that shows the quantity of each product sold, identifying high-demand items.
8. To deliver a modern, responsive user interface with bottom navigation bars, card layouts, search filters, and intuitive forms.

Chapter 3: Introduction

In today's Digital Era, Mobile Applications have become the sole medium for Shopping and Entertainment purposes. People frequently use separate apps for buying products and discovering&booking the events (concerts, workshops, sports, etc.). There is a clear gap in the market where a single platform can be used to both purchase physical goods and reserve seats for local events.

And "E-Shop & Events" bridges this gap. Cause its sole purpose is to create a single platform where a user/seller can both purchase/add the products and for booking/ creating the events.

"E-Shop & Events" is a Flutter-based mobile application that serves two types of users: normal customers and sellers.

Where, A Customer can:

- Register and log in with email/password or via Google Sign-in.
- Browse products, search by name, filter by category.
- Add products to a persistent shopping cart, enter delivery address, pin-code and can use the coupons(to get a discount) and place an order (dummy payment).
- Browse upcoming event, book a seats(if the seats are available) and view/cancel the bookings.
- View order history and available coupons.

And, The Seller (after separate registration requiring business name, address, GST number, and phone number) can:

- Log in and access a dedicated dashboard.
- Add new products (with name, description, price, stock, category, and an image uploaded via Cloudinary).
- Add new events (with title, description, date, location, total seats, and event image).
- View customer orders that contain at least one of his/her products (filtered server-side).

- View product & analyse sales showing total units sold per product. To know which one of his product is in high demand.
- Create discount coupons.(So, the customer can use them)

Its Backend is built on Firebase: “Firestore” for real-time data, Firebase Authentication for user management, and “Cloudinary” for image uploads. The app uses ”SharedPreferences” to keep each user’s cart separate (using the user’s unique ID as part of the storage key).
It is Developed using Flutter framework with a clean architecture separating models, services (Firebase/Cloudinary interactions), and UI screens. The project demonstrates the use of stateful widgets, streams, form validation, and modern UI patterns.

Chapter 4: Problem Statement

I have done some research on small business owners in my locality. Where they said that they use separate apps for selling products (WhatsApp Business, Amazon) and promoting events (Eventbrite, Paytm Insider). They complained about double data entry, missed sales, and confused customers. This inspired me to build a single app that serves both needs..

From a user perspective, individuals are required to install and manage multiple applications to fulfill their needs—one for purchasing products and another for discovering and booking events. This leads to increased device storage usage, repeated account registrations, inconsistent user interfaces, and fragmented data such as order history and bookings. Additionally, users lack a unified experience where they can seamlessly switch between shopping and event-related activities using a single platform.

From a seller or service provider perspective, small business owners and event organizers face significant challenges in maintaining separate systems for product sales and event management. Managing multiple platforms increases operational complexity, requires duplicate data entry, and often involves additional costs for subscriptions, hosting, and maintenance. Moreover, many available solutions are either web-centric, expensive, or technically complex, making them less accessible for small-scale entrepreneurs and startups.

Another critical issue is the lack of mobile-first, cost-effective, and scalable solutions that combine these functionalities while still offering essential features such as secure authentication, role-based access, real-time data updates, and media management. Many existing platforms also lack customization options, integrated analytics, and efficient resource handling, which are important for modern digital businesses.

Furthermore, users increasingly expect real-time interaction, intuitive interfaces, and personalized experiences, which are not adequately provided when services are split across multiple systems. The absence of integrated analytics also prevents sellers from gaining insights into customer behavior, product demand, and event popularity.

Therefore, the core problem addressed in this project is the design and development of a unified, mobile-first application that integrates both e-commerce and event management functionalities into a single platform.

The system must:

4.1. Provide role-based access control for customers and sellers.

4.2. Support product browsing, search, filtering, cart management, and order placement.

4.3. Enable event listing, seat selection, booking, and cancellation.

4.4. Offer efficient image handling for products, events, and user profiles.

4.5. Maintain data consistency and persistence using a real-time backend.

4.6. Deliver a user-friendly and responsive interface for improved usability.

4.7. Include basic analytics and reporting tools for sellers.

The challenge lies in implementing all these features in a scalable, secure, and cost-effective manner using modern technologies such as Flutter and Firebase, while ensuring that both functional and non-functional requirements (performance, usability, reliability, and maintainability) are satisfied.

This project aims to address these gaps by providing a comprehensive, integrated solution that simplifies digital operations for sellers and enhances convenience for users, ultimately improving efficiency, accessibility, and overall user experience.

Chapter 5: Software Requirements Specification (SRS)

5.1 UML Use case diagram:

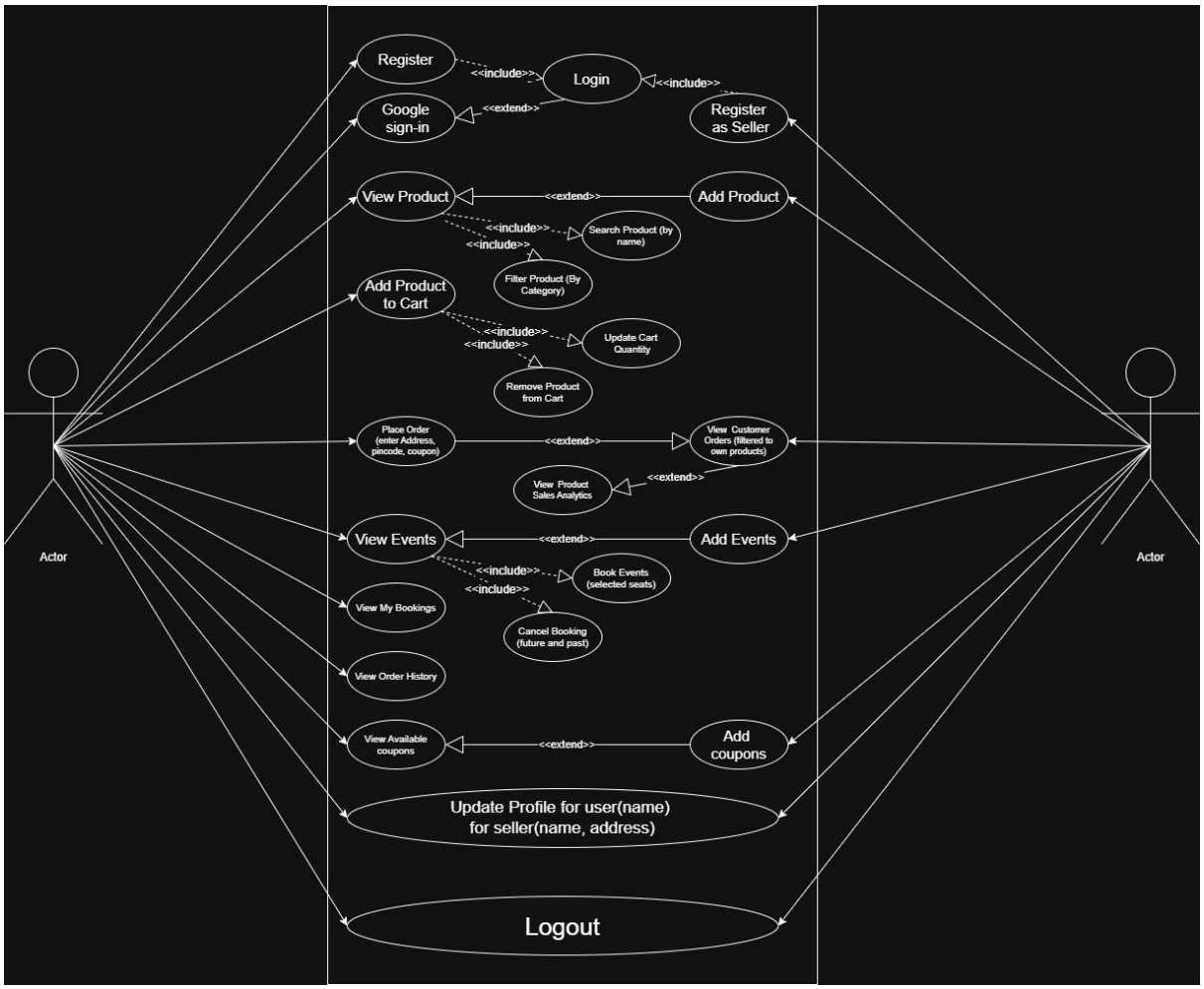


Fig.5.1

5.2 Use Case Specifications:

5.2.1 Add Product:

Use Case Name	Add Product
Pre- condition	Seller is authenticated and logged into the system.
Basic Path	1. Seller navigates to the Dashboard tab and taps the "Add Product" card. 2. System displays the Add Product form. 3. Seller enters product name, description, price, stock. 4. Seller selects a category from the dropdown. 5. Seller taps the image container to pick an image from the gallery. 6. System shows the selected image preview. 7. Seller clicks "Add Product". 8. System uploads the image to Cloudinary and receives a secure URL. 9. System stores all product data (including image URL and category) in Firestore. 10. System shows a success message and returns to the seller dashboard.
Alternative Path	5.1. Seller cancels image selection – no image is uploaded, and the product cannot be submitted (validation fails).
Post- condition	The new product document appears in the Firestore products collection and becomes visible to all users in the Products list.
Exception Path	– Cloudinary upload fails → system shows "Image upload failed", product is not saved. – Missing required field → validation error message, product not saved. – Firestore write fails → appropriate error message is displayed.

5.2.2 Book Event:

Use Case Name	Book Event
Pre- condition	Normal user is authenticated. Event exists with available seats > 0.
Basic Path	1. User navigates to the Events tab and sees the list of upcoming events (cards). 2. User taps "View & Book" on a chosen event. 3. System shows the event detail screen with description, date, location, total seats, available seats. 4. User selects the number of seats (using +/– buttons). 5. User taps "Book Now". 6. System creates a new booking document in Firestore containing eventId, userId, eventTitle, eventDate, location, seatsBooked, and bookedAt timestamp. 7. System decrements the available seats of the event by the booked quantity. 8. System shows a success message and returns to the Events list.
Alternate Path	4.1. User tries to book more seats than available → the "+" button is disabled, and a warning is shown when trying to exceed.
Post- condition	The booking is recorded, and the event's available seats are updated. The user can see the booking in the "My Bookings" screen.
Exception Path	– Firestore write fails (e.g., network error) → system displays "Booking failed" with the error message. – Unauthenticated user attempts to book → system shows "Please login first".

5.2.3 Place Order:

Use Case Name	Place Order
Pre-condition	User is authenticated and has at least one item in the cart.
Basic Path	1. User goes to Cart screen and verifies items. 2. User taps "Proceed to Checkout". 3. System displays checkout form with fields: Address, Pincode, Coupon Code (optional), Total amount. 4. User fills in address and pincode (must be 6 digits) and may enter a coupon code. 5. User taps "Place Order (Dummy Payment)". 6. System validates coupon (if entered) and applies discount. 7. System creates an order document in Firestore with fields: userId, items list, totalAmount (after discount), address, pincode, orderId (generated UUID), createdAt timestamp, and phone number of the user. 8. System clears the user's cart (via SharedPreferences). 9. System navigates to the Order Confirmation screen showing the generated order ID.
Alternate Path	4.1. User leaves address or pincode empty – validation fails, order not placed. 4.2. Pincode not exactly 6 digits – validation fails. 4.3. Coupon invalid or expired – error message shown.
Post-condition	Order is stored permanently, cart is empty, and user receives an order ID.
Exception Path	– Firestore add operation fails → error message shown, cart remains unchanged, loading indicator stops.

Chapter 6: Cost / Effort Estimation

6.1. Monetary Cost:

- All technologies used are free for development and small- scale usage:
- Flutter: Open- source SDK.
- Firebase (Spark plan): Free quota includes 1 GB Firestore storage, 10 GB/month network, 5 GB Cloud Storage (Firebase Storage not used, but Firestore is within free tier).
- Cloudinary (free tier): 25+ GB storage, 25+ GB monthly bandwidth – sufficient for a college project.
- Development tools: Android Studio, VS Code (free).
- Hosting: Not required – app is installed locally on devices.
- Total monetary cost = ₹0 / $0

6.2. Effort Estimation:

Phase	Duration (weeks)
Requirements analysis & planning	1
UI design (wireframes, prototypes)	1
Firebase project setup & backend design	0.5
Flutter development (frontend + backend integration)	4
Image handling (Cloudinary integration)	0.5
Testing and bug fixing	2
Documentation & report writing	2
Total	11 weeks

Chapter 7: UML Class Diagram

The class diagram includes the main data models used in the application. All classes are part of the models folder.

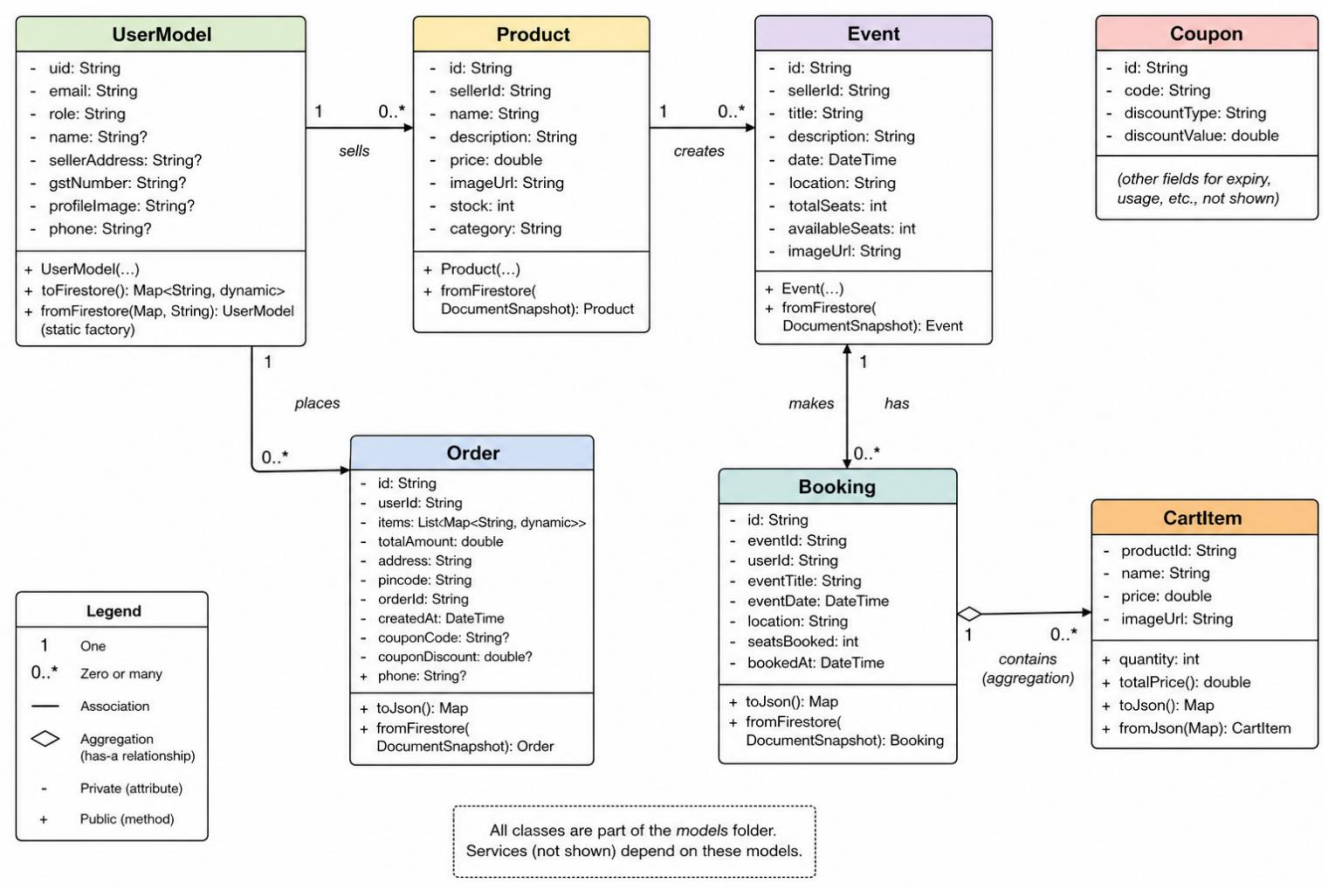


Fig.7.1

7.1. Attribute:

7.1.1. UserModel:

7.1.1.1. - uid: String

7.1.1.2. - email: String

7.1.1.3. - role: String

7.1.1.4. - name: String?

7.1.1.5. - sellerAddress: String?

7.1.1.6. - gstNumber: String?

7.1.1.7. - profileImage: String?

7.1.1.8. - phone: String?

+ UserModel(...) constructor

+ toFirestore(): Map<String, dynamic>

+ fromFirestore(Map, String): UserModel (static factory)

7.1.2. Product:

7.1.2.1. - id: String

7.1.2.2. - sellerId: String

7.1.2.3. - name: String

7.1.2.4. - description: String

7.1.2.5. - price: double

7.1.2.6. - imageUrl: String

7.1.2.7. - stock: int

7.1.2.8. - category: String

+ Product(...)

+ fromFirestore(DocumentSnapshot): Product

7.1.3. Event:

7.1.3.1. - id: String

7.1.3.2. - sellerId: String

7.1.3.3. - title: String

7.1.3.4. - description: String

7.1.3.5. - date: DateTime

7.1.3.6. - location: String

7.1.3.7. - totalSeats: int

7.1.3.8. - availableSeats: int

7.1.3.9. - imageUrl: String

+ Event(...)

+ fromFirestore(DocumentSnapshot): Event

7.1.4. Order:

7.1.4.1. - id: String

7.1.4.2. - userId: String

7.1.4.3. - items: List<Map<String, dynamic>>

7.1.4.4. - totalAmount: double

7.1.4.5. - address: String

7.1.4.6. - pincode: String

7.1.4.7. - orderId: String

7.1.4.8. - createdAt: DateTime

7.1.4.9. - couponCode: String?

7.1.4.10. - couponDiscount: double?

7.1.4.11. - phone: String?

+ toJson(): Map

+ fromFirestore(DocumentSnapshot): Order

7.1.5. Booking:

7.1.5.1. - id: String

7.1.5.2. - eventId: String

7.1.5.3. - userId: String

7.1.5.4. - eventTitle: String

7.1.5.5. - eventDate: DateTime

7.1.5.6. - location: String

7.1.5.7. - seatsBooked: int

7.1.5.8. - bookedAt: DateTime

+ toJson(): Map

+ fromFirestore(DocumentSnapshot): Booking

7.1.6. CartItem:

7.1.6.1. - productId: String

7.1.6.2. - name: String

7.1.6.3. - price: double

7.1.6.4. - imageUrl: String

7.1.6.5. + quantity: int

+ totalPrice(): double

+ toJson(): Map

+ fromJson(Map): CartItem

7.1.7. Coupon:

7.1.7.1. - id: String

7.1.7.2. - code: String

7.1.7.3. - discountType: String

7.1.7.4. - discountValue: double

7.2. Associations :

- UserModel ————> Order
- UserModel ————> Booking
- Event ————> Booking
- UserModel (Seller) ————> Product
- UserModel (Seller) ————> Event

7.3. Aggregation:

- Order ———> CartItem
  - (Order contains multiple CartItems)

7.4. Dependency:

- Services layer (not shown) depends on all model classes

Chapter 8: Entity Relationship (ER) Diagram

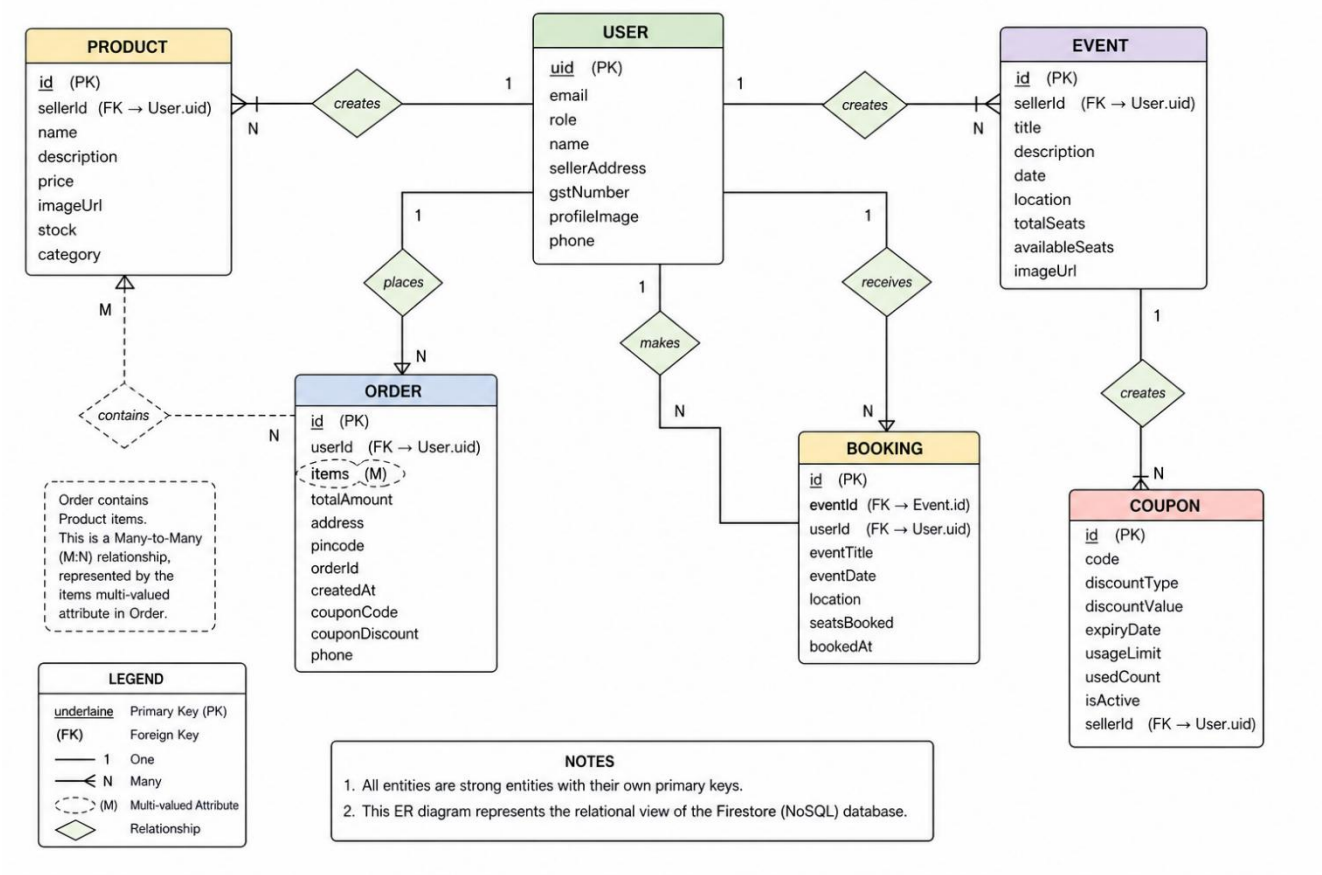


Fig. 8.1

The ER diagram is drawn for the Firestore NoSQL database. However, for relational representation, the following entities are identified:

8.1. Entities (strong entities, each with a unique identifier as primary key):

8.1.1. User (PK: uid)

8.1.2. Product (PK: id)

8.1.3. Event (PK: id)

8.1.4. Order (PK: id)

8.1.5. Booking (PK: id)

8.1.6. Coupon (PK: id)

8.2. Weak entities: None; every entity has its own primary key.

8.3. Attributes (primary keys are underlined in the diagram):

8.3.1. User: uid, email, role, name, sellerAddress, gstNumber, profileImage, phone

8.3.2. Product: id, sellerId (foreign key to User), name, description, price, imageUrl, stock, category

8.3.3. Event: id, sellerId (FK to User), title, description, date, location, totalSeats, availableSeats, imageUrl

8.3.4. Order: id, userId (FK to User), items (multi-valued attribute), totalAmount, address, pincode, orderId, createdAt, couponCode, couponDiscount, phone

8.3.5. Booking: id, eventId (FK to Event), userId (FK to User), eventTitle, eventDate, location, seatsBooked, bookedAt

8.3.6. Coupon: id, code, discountType, discountValue, expiryDate, usageLimit, usedCount, isActive, sellerId (FK to User)

8.4. **Relationships and Cardinalities:**

8.4.1. User (seller) creates Product – one-to-many (1:N).

8.4.2. User (seller) creates Event – one-to-many (1:N).

8.4.3. User (customer) places Order – one-to-many (1:N).

8.4.4. User (customer) makes Booking – one-to-many (1:N).

8.4.5. Event receives Booking – one-to-many (1:N).

8.4.6. Order contains Product items – many-to-many (M:N) expressed by the items array attribute in the Order entity.

8.4.7. User (seller) creates Coupon – one-to-many (1:N).

Chapter 9: Sequence Diagram for “Book Event”

This sequence diagram illustrates the interaction between the user, UI components, services, and Firestore when a user books seats for an event.

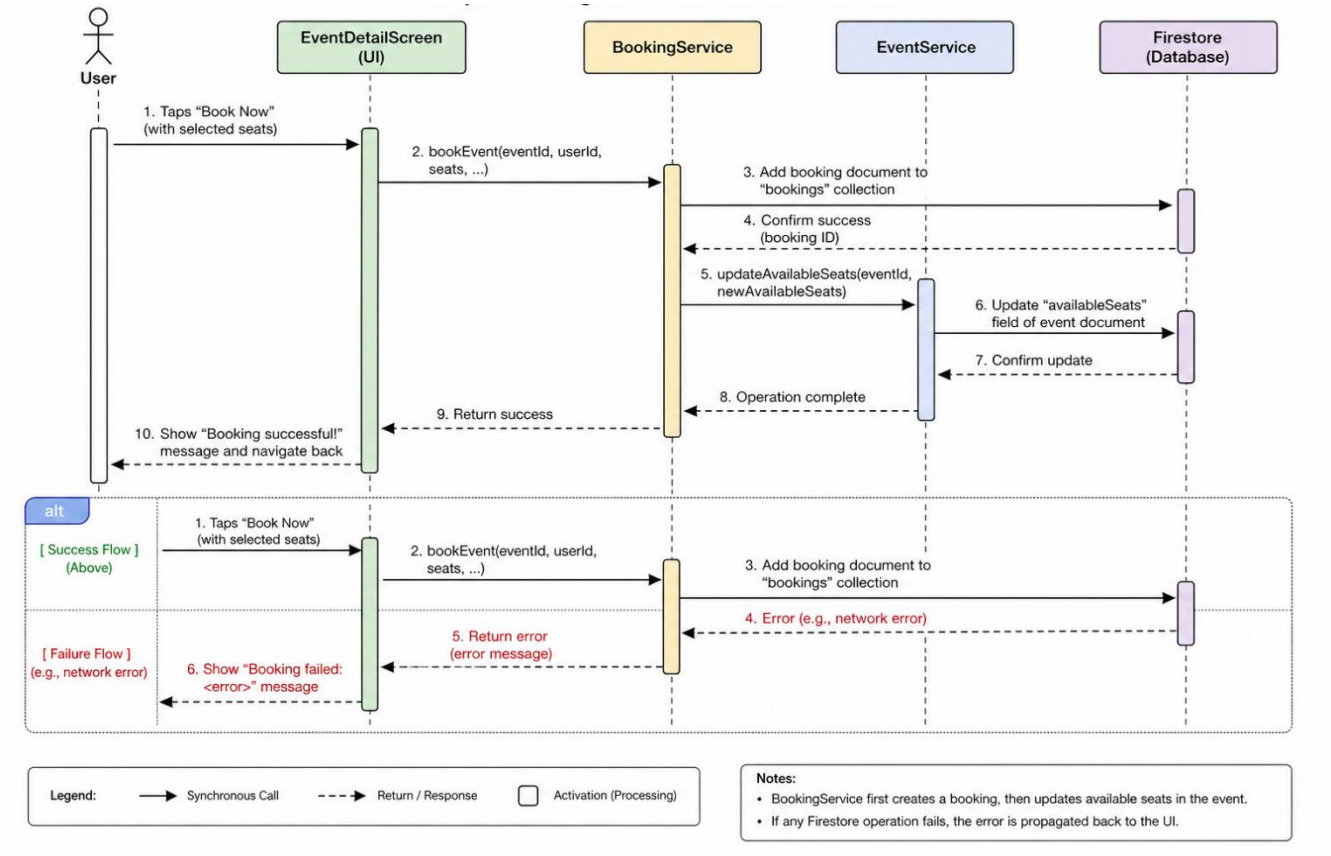


Fig.9.1

9.1. Lifelines:

9.1.1. User (actor)

9.1.2. EventDetailScreen (UI)

9.1.3. BookingService

9.1.4. EventService

9.1.5. Firestore (database)

9.2. Messages (steps):

9.2.1. User → EventDetailScreen: “Taps Book Now” (with selected seats).

9.2.2. EventDetailScreen → BookingService: bookEvent(eventId, userId, seats, ...)

9.2.3. BookingService → Firestore: add booking document to bookings collection.

9.2.4. Firestore → BookingService: confirm success (booking ID).

9.2.5. BookingService → EventService: updateAvailableSeats(eventId, newAvailableSeats)

9.2.6. EventService → Firestore: update the availableSeats field of the event document.

9.2.7. Firestore → EventService: confirm update.

9.2.8. EventService → BookingService: operation complete.

9.2.9. BookingService → EventDetailScreen: return success.

9.2.10. EventDetailScreen → User: show “Booking successful!” message and navigate back.

9.3. Alternative flows:

If Firestore operation fails (e.g., network error), an error response is sent back and the UI shows “Booking failed: <error>”.

Chapter 10: Descriptions of Different Input Validations and Checks

The application implements the following validations:

Screen / Field	Validation Rule	Error Message / Behaviour
Login – Email	Cannot be empty; must be a valid email format (Firebase validates)	"Login failed" if invalid.
Login – Password	Cannot be empty	"Login failed".
Customer Registration – Name, Email, Phone, Password	All fields required	"Required" shown under each field.
Seller Registration – Business Name, Email, Phone, Password, Address, GST	All required	Form validation; "Required" shown.
Add Product – Name, Desc, Price, Stock, Category, Image	All required	"Fill all fields and select image".
Add Product – Price, Stock	Must be numeric, >0	KeyboardType.number; validator ensures not empty.
Add Event – Title, Desc, Location, Seats, Date, Image	All required; seats >0	Similar validation.

Screen / Field	Validation Rule	Error Message / Behaviour
Checkout – Address	Not empty	"Enter address".
Checkout – Pincode	Exactly 6 digits	"Enter valid pincode".
Checkout – Coupon Code	Code must exist, not expired, within usage limit	"Invalid or expired coupon".
Book Event – Seats	Between 1 and available seats	Buttons disabled when limit reached; error shown if trying to exceed.
Cancel Booking	Confirmation dialog before deletion	"Are you sure?" with Yes/No.
Profile – Name update	Not empty (onFieldSubmitted)	Name updated silently.
Image selection	Only image files (gallery restriction)	Image picker shows only images.
Login after logout	None	Redirects to login screen.
Coupon creation (seller)	Code, type, value, limit, expiry date all required	Validation errors shown.

Chapter 11: Description of Different Reports

The application does not generate printable PDF reports but provides on-screen reports in the form of dedicated screens:

11.1. Customer Orders (Seller Report):

11.1.1. Access: Seller Dashboard → "Customer Orders".

11.1.2. Shows all orders that contain at least one product belonging to the logged-in seller.

11.1.3. Displays order ID, total amount, user ID, delivery address, user phone number, and only the items that the seller sold.

11.1.4. Allows expanding each order to view details.

11.2. Product Sales Analytics (Seller Report):

11.2.1. Access: Seller Dashboard → "Product Sales".

11.2.2. Lists each of the seller's products, with the total quantity sold (based on all orders).

11.2.3. Sorted from most sold to least sold, helping the seller identify high-demand items.

11.2.4. Includes product image, name, price, and sold units.

11.3. My Bookings (User Report):

11.3.1. Access: Bottom navigation → "My Bookings".

11.3.2. Shows all events the user has booked, with date, location, number of seats, booking date.

11.3.3. Past events are visually distinguished (grey icon, "Event passed" label).

11.3.4. Provides a delete button to cancel any booking (future bookings increase available seats).

11.4. Order Confirmation (User Receipt):

11.4.1. Shown immediately after placing an order.

11.4.2. Displays the unique order ID (generated UUID).

11.4.3. Acts as a proof of purchase.

11.5. Order History (User Report):

11.5.1. Access: Profile → "View Order History".

11.5.2. Lists all orders placed by the logged- in user, with order ID, total amount, date, address, items, and any coupon used.

11.5.3. Expandable to show details.

11.6. Available Coupons (User Report):

11.6.1. Access: Profile → "Available Coupons".

11.6.2. Lists all active coupons (not expired, within usage limit) with discount details and remaining uses.

11.6.3. Tapping a coupon shows a snackbar with the code to copy.

Chapter 12: Sample Test Cases

12.1. White- box Test Cases:

12.1.1. Module 1: addToCart method of CartService

The method has a single decision (if product already in cart).

Cyclomatic complexity = 2.

Flow Graph:

Node A: Start → Node B: Check index >=0?

True → Node C: Update quantity.

False → Node D: Add new item.

Both converge to Node E: Save cart.

Independent paths:

A -> B -> C -> E

A -> B -> D -> E

Test cases:

Test ID	Description	Input	Expected Output
W1	Add product not in cart	New product	Cart length increases by 1; product quantity = 1.
W2	Add product already in cart	Existing product	Quantity increments; cart length unchanged.

Execution results: Both passed.

12.1.2. Module 2: bookEvent method (client- side validation inside UI):

The method includes seat limit check and database write. Cyclomatic complexity = 3 (if seats <= available, try- catch for write).

Independent paths:

- Happy path: valid booking → booking added, seats decreased.
- Firestore write failure → catch block executed, error shown.
- Seat overflow (client- side) → not allowed.

Test cases:

Test ID	Description	Input	Expected Output
W3	Normal booking	valid event, seats within limit	Booking document created; event seats reduced.
W4	Firestore exception (simulated by network disconnection)	normal request	Error message “Booking failed: [error]”.

Execution results: Both passed.

12.2. Black- box Test Cases:

Use Case: Login (Equivalence Class Partitioning)

Input domain: email and password. Classes:

- Valid email + valid password
- Invalid email format
- Valid email + wrong password
- Non- existent user

Test ID	Input Email	Input Password	Expected Result
B1	correct@example.com	correct123	Redirect to home screen.
B2	invalid-email	any	“Login failed” (Firebase error).
B3	correct@example.com	wrongpass	“Login failed”.
B4	notexist@example.com	any	“Login failed”.

Use Case: Place Order (Boundary Value Analysis)

Focus on pincode field (minimum length = 6, maximum length = 6). Boundaries: 5, 6, 7 digits.

Test ID	Pincode	Address	Expected Result
B5	12345 (5 digits)	valid	Validation error: “Enter valid pincode”.
B6	123456 (6 digits)	valid	Order placed successfully.
B7	1234567 (7 digits)	valid	Validation error.

Execution results: All black- box test cases passed.

Chapter 13: Snapshots of Different Input and Output Screens

Fig.13.1. Registration Screen(User/Seller):

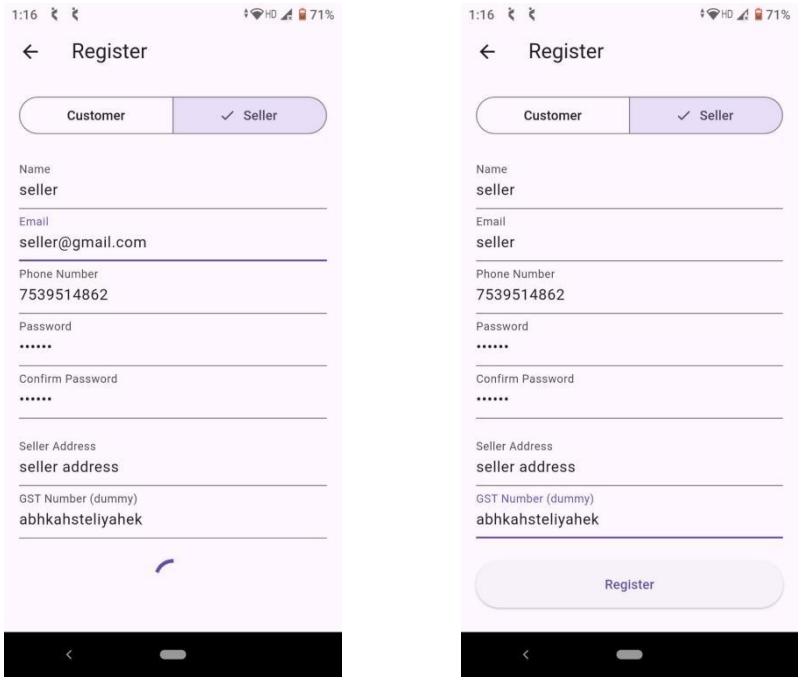


Fig.13.2. Login Screen(User/Seller):

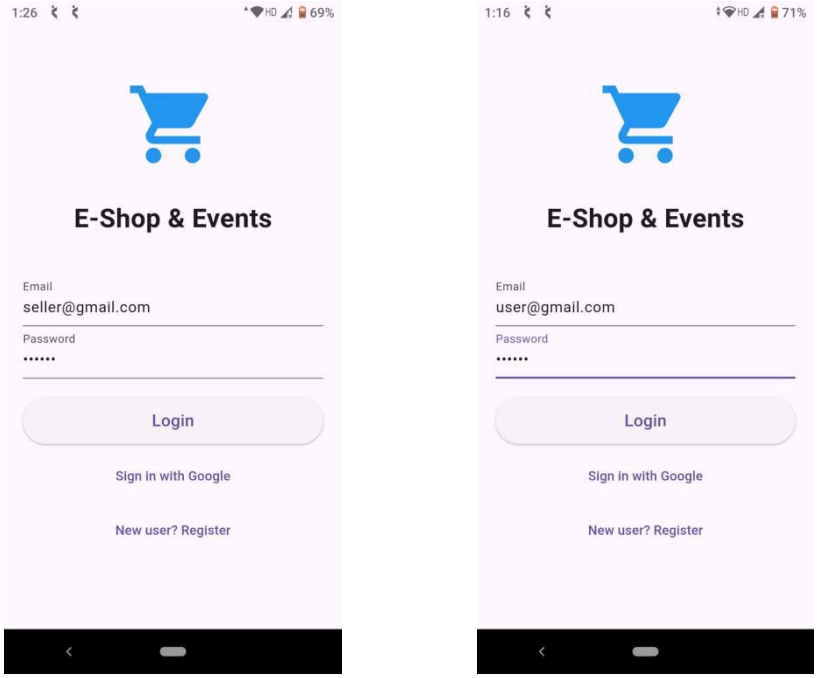
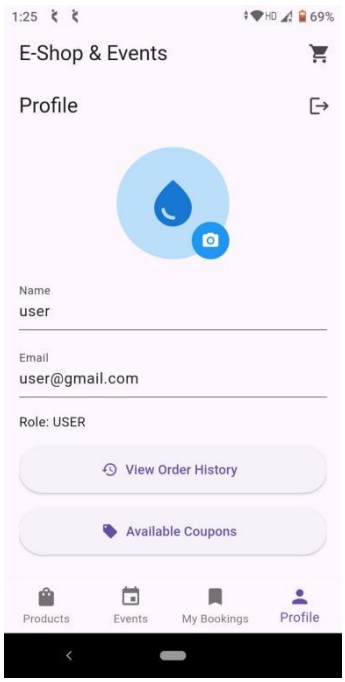
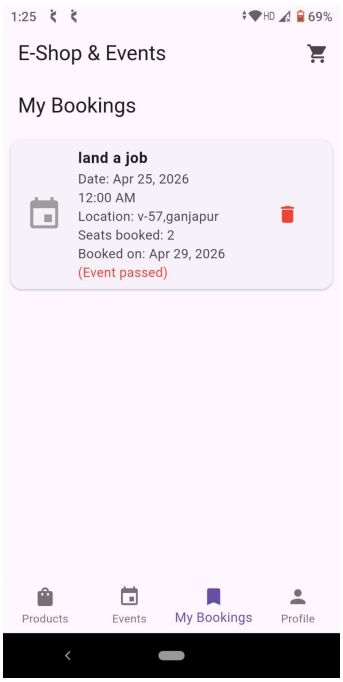
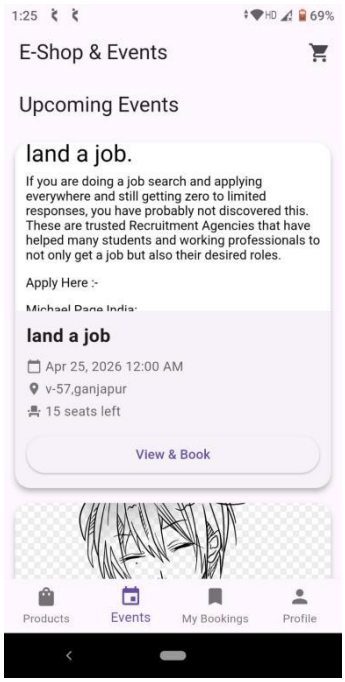


Fig13.3. User Screens(Bottom navigation: Products, Events, My Bookings, Profile):







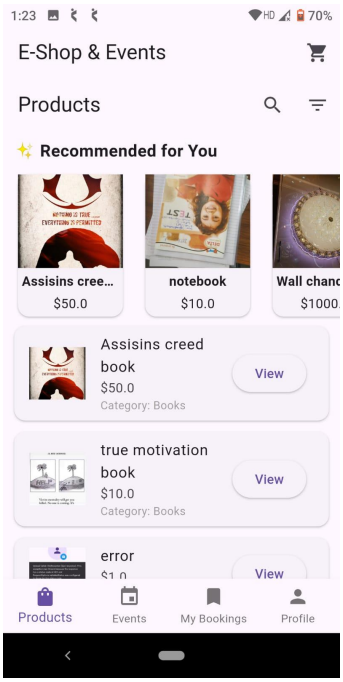
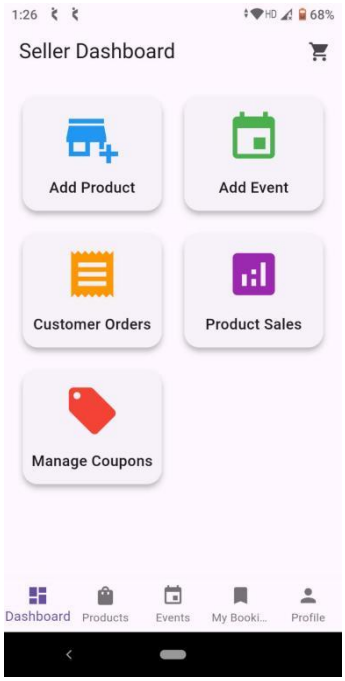
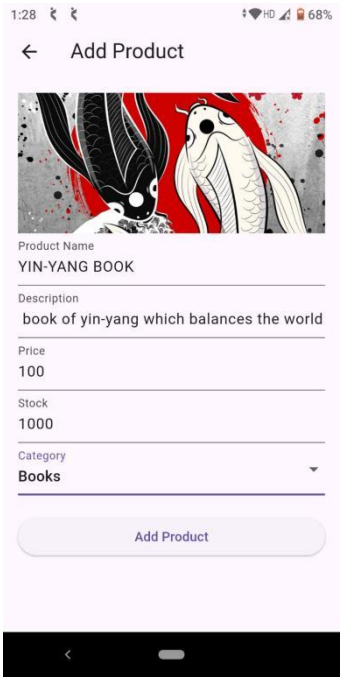
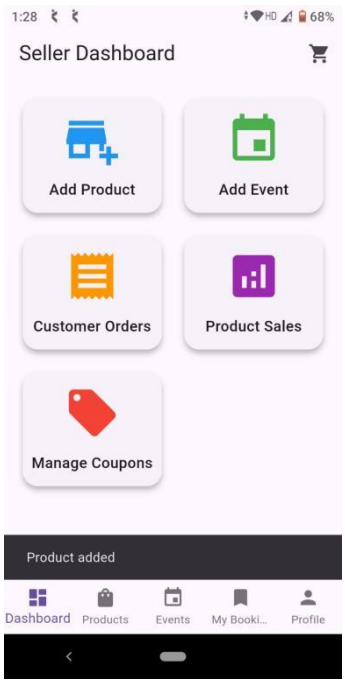
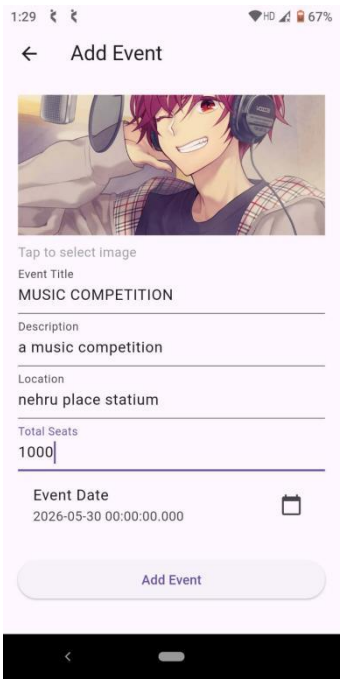


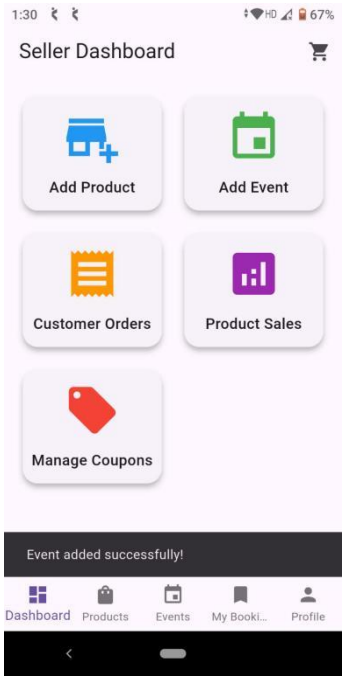
Fig.13.4.Seller Dashboard (Grid of cards: Add Product, Add Event, Customer Orders, Product Sales, Manage Coupons):





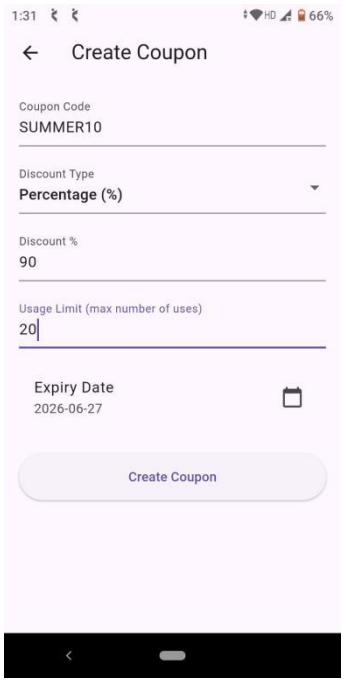





1:30
67%
Seller Dashboard
Add Product
Add Event
Customer Orders
Product Sales
Manage Coupons
Event added successfully!
Dashboard
Products
Events
My Booki...
Profile


1:31
66%
Customer Orders
Order ID: f95d3110-d8e9-4d6e-a26d-6201845b4eb2
Total: $50.0


1:31
66%
Product Sales Analytics
YIN-YANG BOOK
Sold: 1 units
$100.0


1:31
66%
Create Coupon
Coupon Code
SUMMER10
Discount Type
Percentage (%)
Discount %
90
Usage Limit (max number of uses)
20
Expiry Date
2026-06-27
Create Coupon

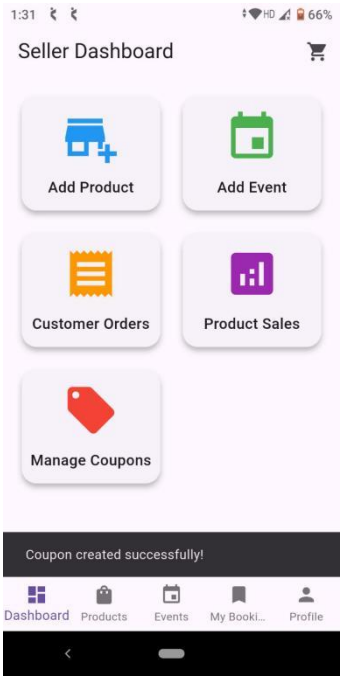

1:31
66%
Seller Dashboard
Add Product
Add Event
Customer Orders
Product Sales
Manage Coupons
Coupon created successfully!
Dashboard
Products
Events
My Booki...
Profile

Fig.13.5. Products Screen (AppBar with search and filter icons, collapsible “Recommended for You” horizontal list, main product list):

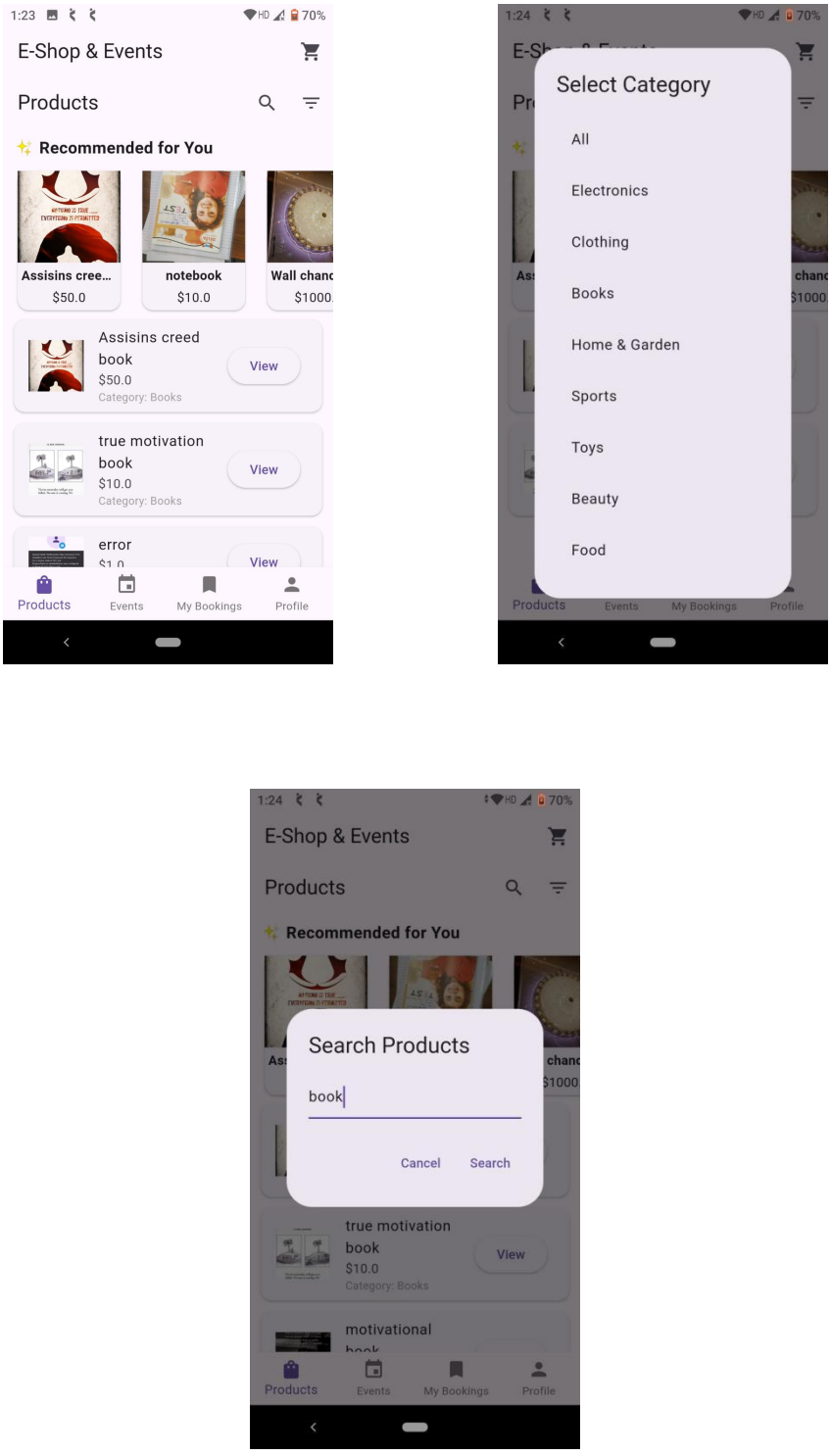
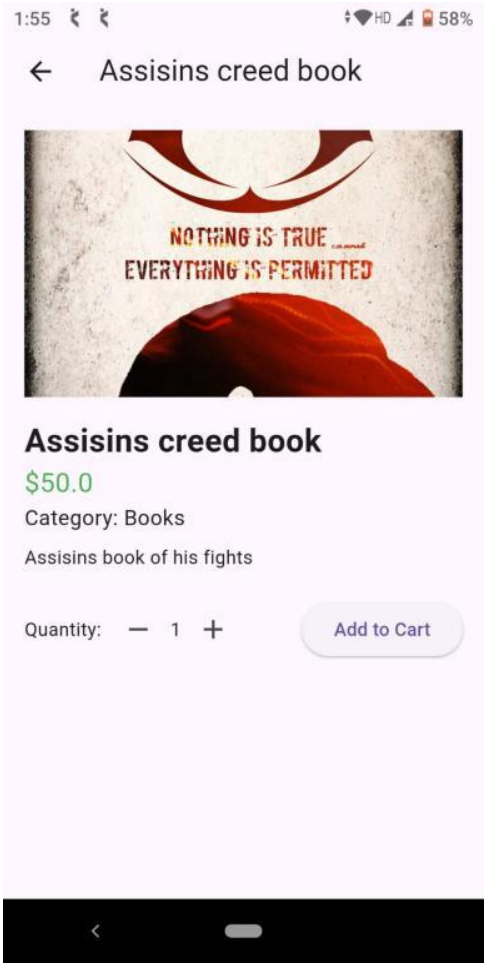


Fig.13.6. Product Detail Screen (Image, name, price, description, quantity selector, “Add to Cart” button) And Cart Screen (List of cart items with quantity controls and delete icon, total amount, “Proceed to Checkout” button):



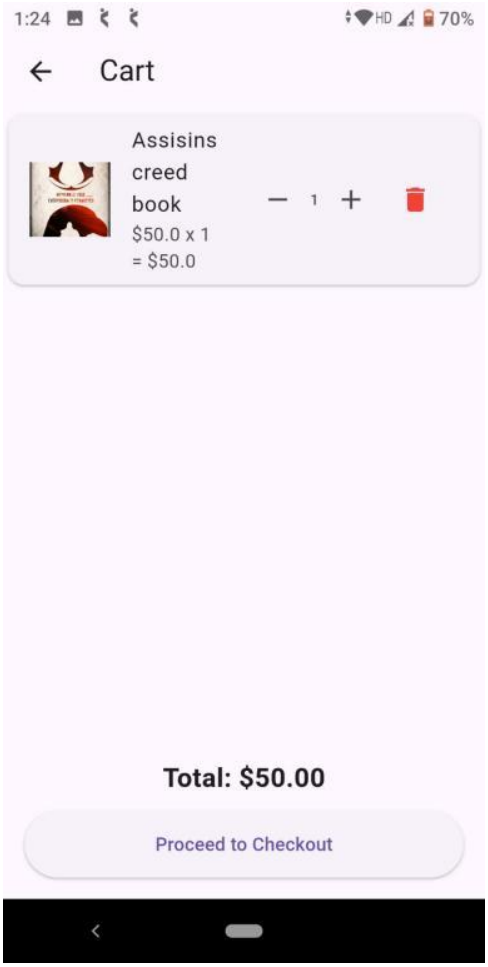
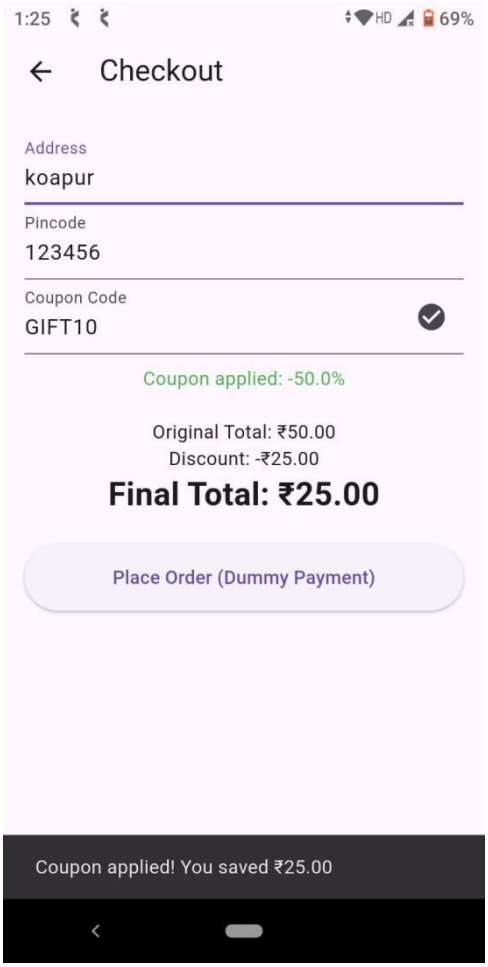


Fig13.7. Checkout Screen (Address, pincode, coupon code field, original total, discount, final total, “Place Order” button) And Order Confirmation Screen (Green checkmark, “Order Placed Successfully!”, order ID, “Back to Home”):



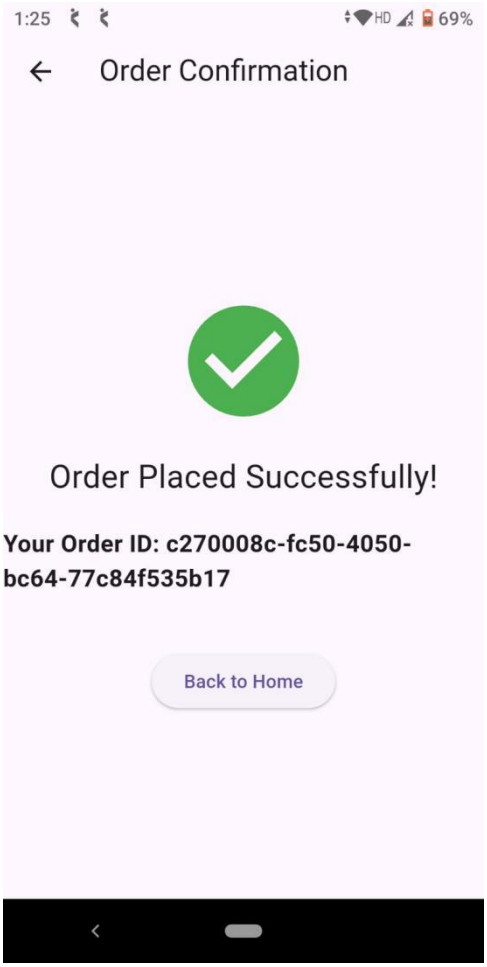
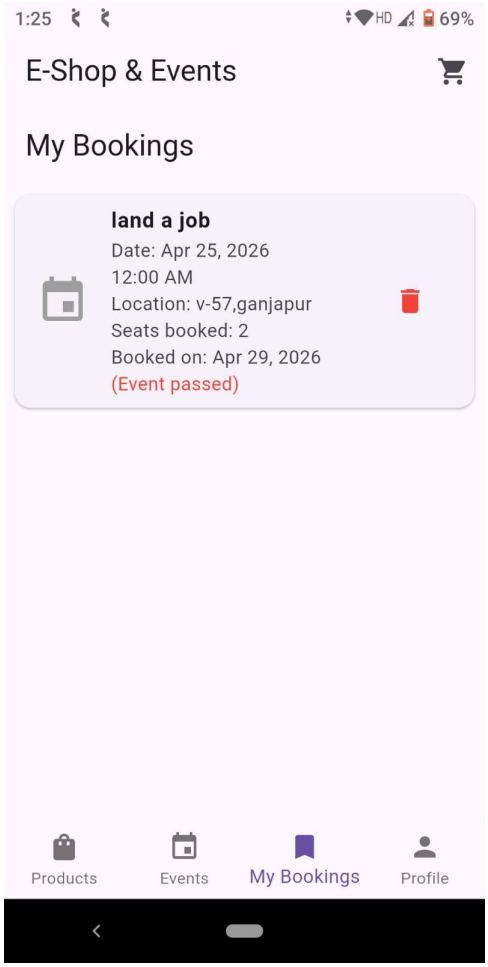


Fig.13.8. Event Detail Screen (Image, description, date, location, seat selector, “Book Now” button) And My Bookings Screen (List of user’s bookings, each with delete icon, past events greyed out):



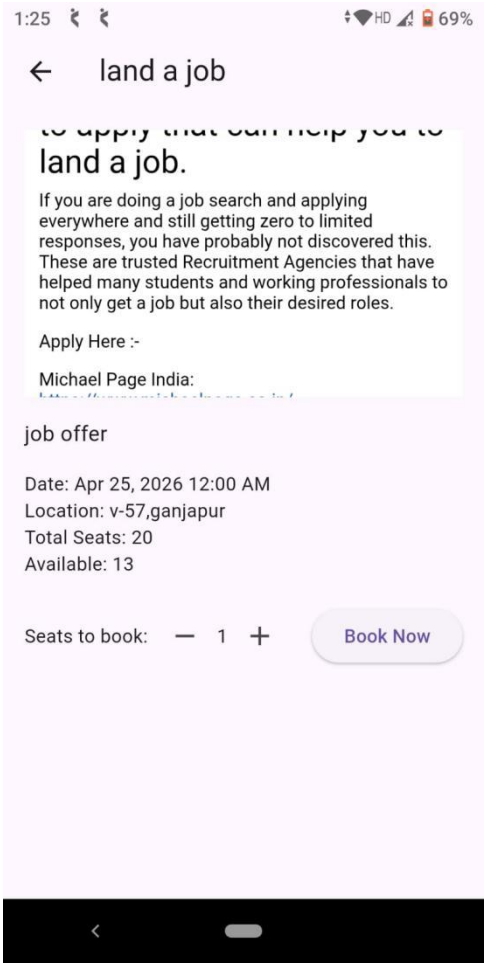
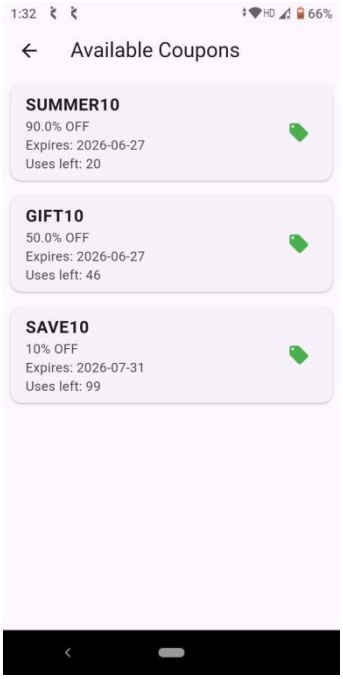


Fig.13.9. Profile Screen, Order History Screen And Availabe Coupon Screen:



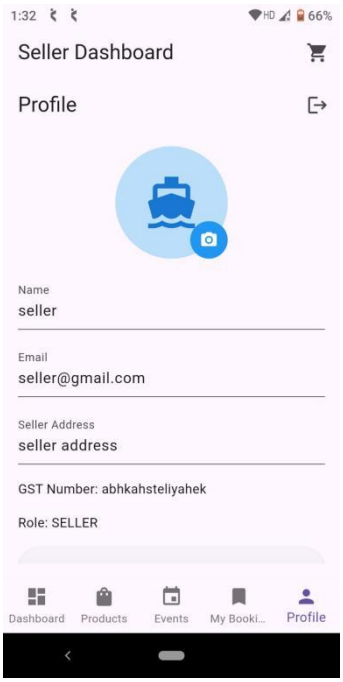


Fig.13.10. Firebase Authentication:

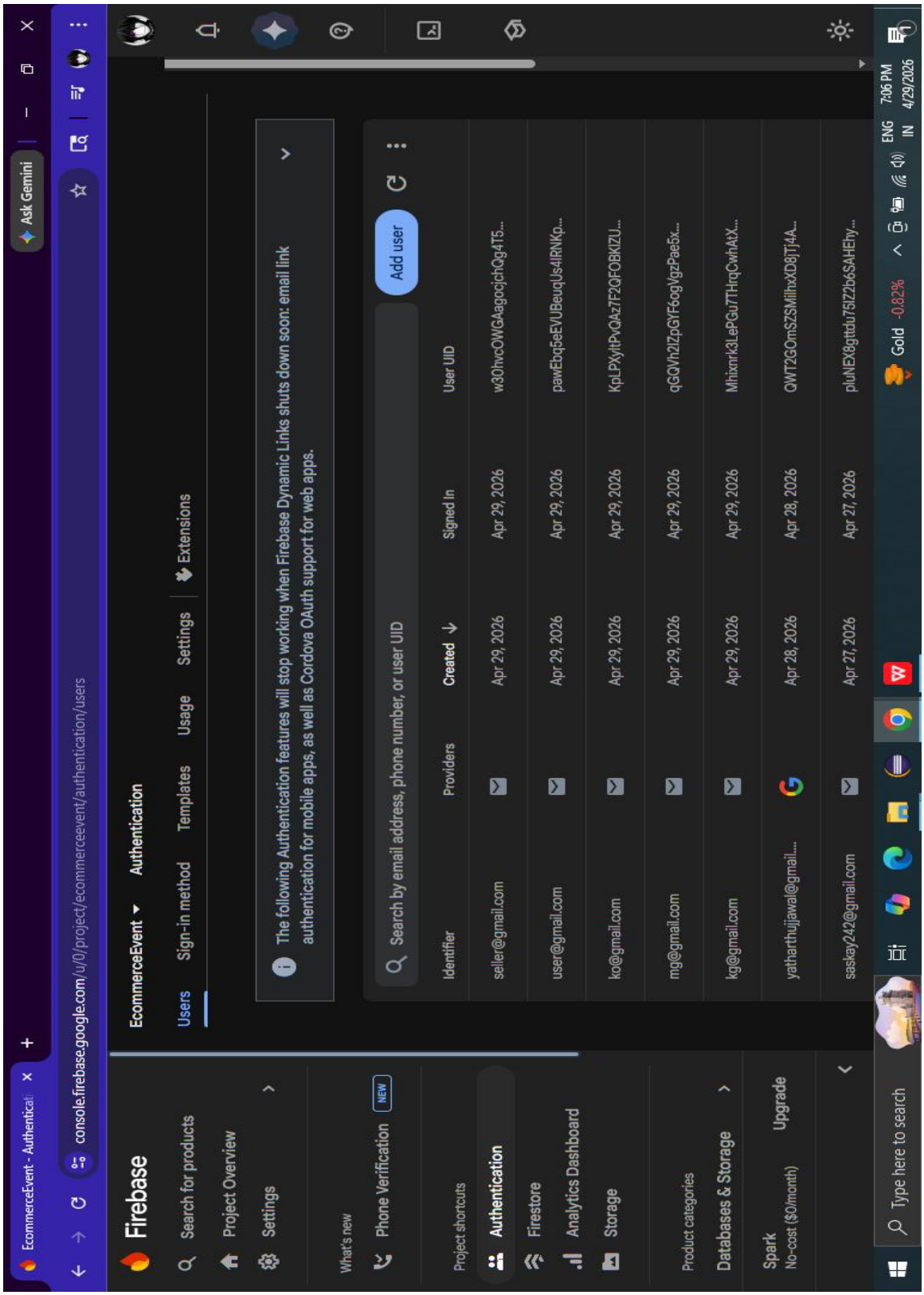


Fig.13.11. Firebase Firestore:

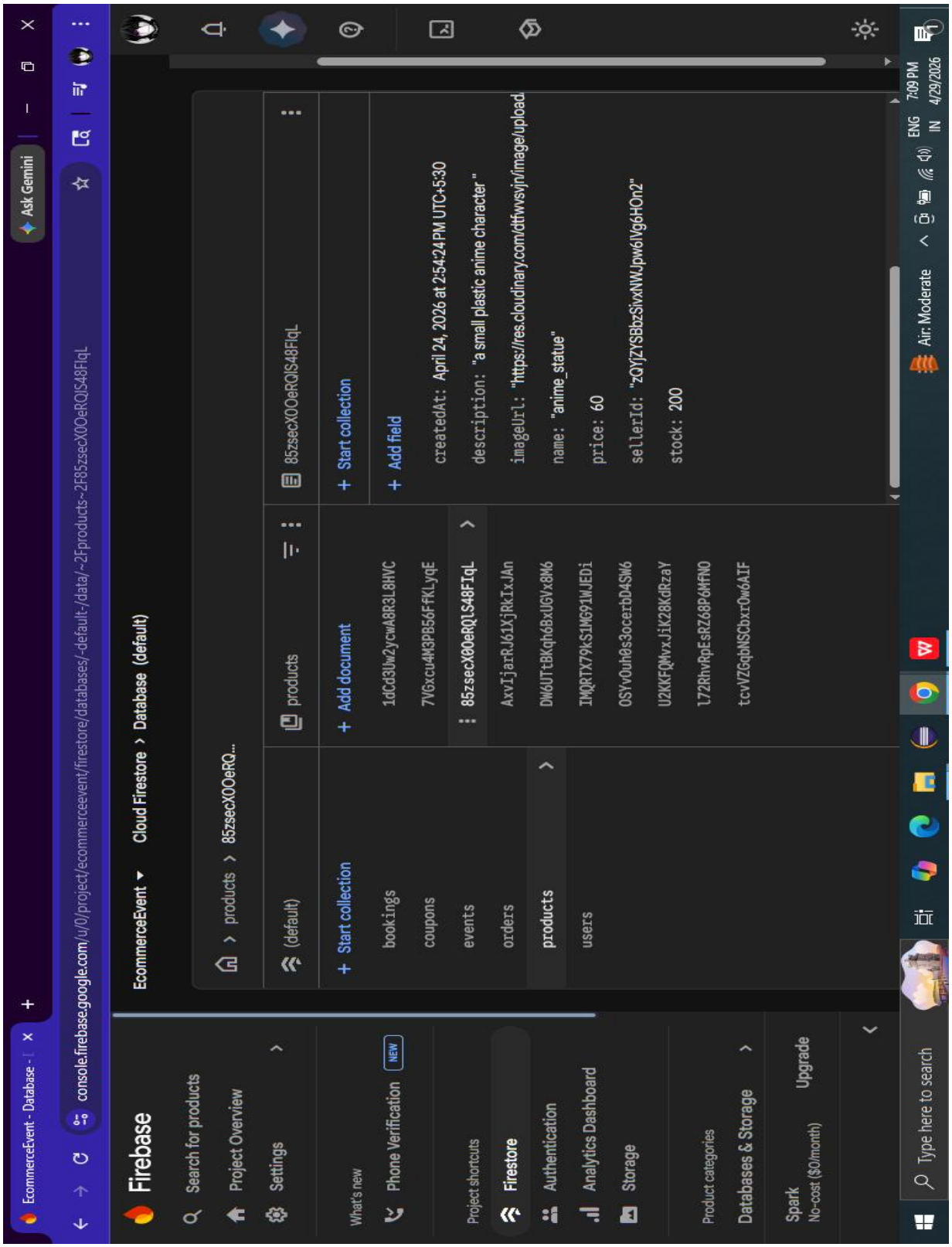
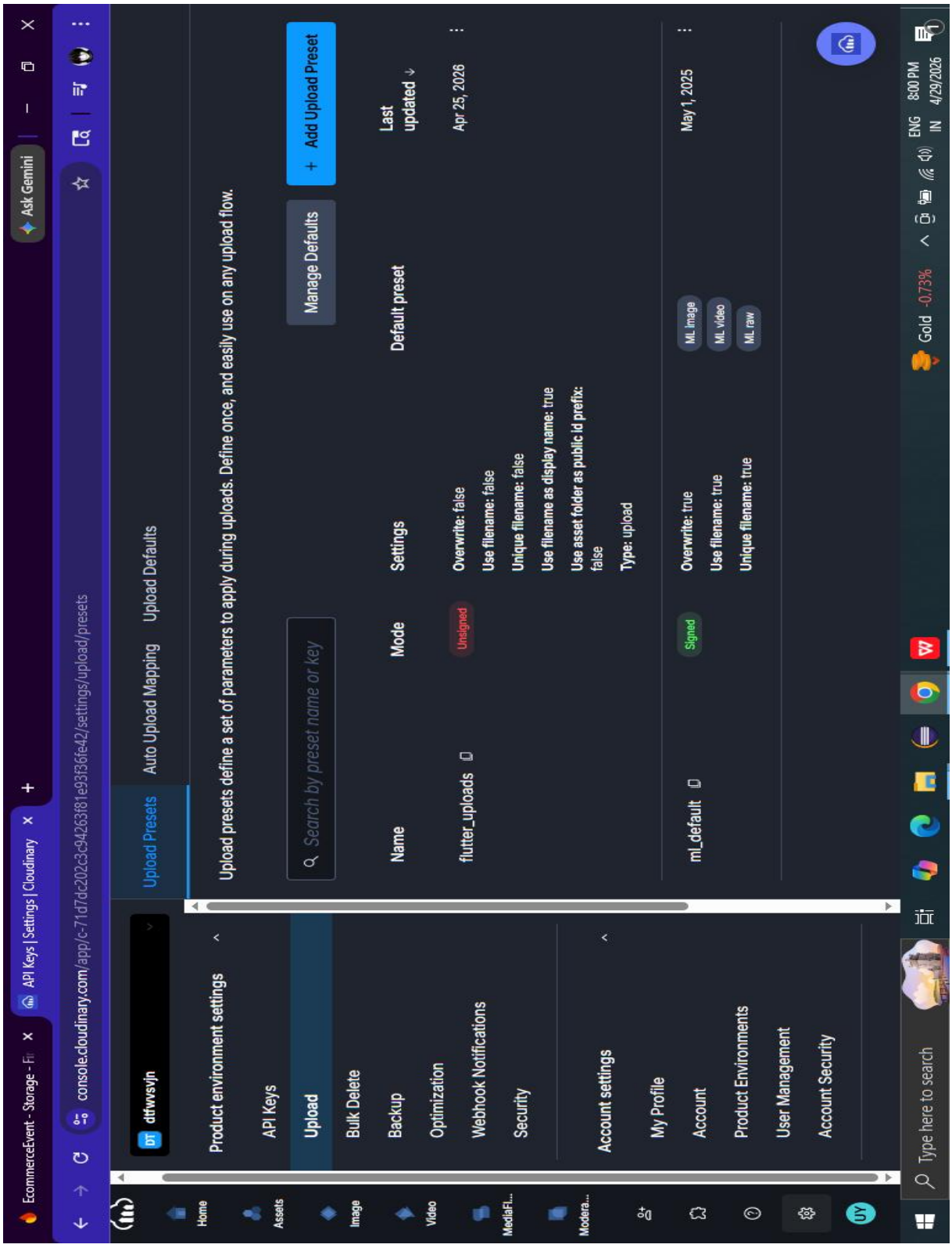


Fig.13.12. Cloudinary:



Chapter 14: Conclusion

The E-Shop & Events mobile application represents a successful attempt at designing and implementing a unified digital platform that integrates two major domains—e-commerce and event management—into a single system.The most challenging part, that I faced was maintaining cart persistence across logins – solved using SharedPreferences with user-specific keys. Thus cause of this project I came to know that Firebase alone cannot handle complex queries & that for the future work, I will add a search index.

Throughout the development process, the application has achieved all its primary objectives. It supports two distinct user roles—customers and sellers—through a well-defined role-based access control mechanism. Customers are able to browse products, apply search and category filters, manage a persistent shopping cart, place orders with proper input validation, and explore as well as book events with seat selection and cancellation options. Sellers, on the other hand, are provided with tools to add and manage products and events, monitor customer orders, create promotional coupons, and analyze product sales through a dedicated analytics interface.

One of the key strengths of this system lies in its integration of modern technologies. The use of Flutter enables the development of a cross-platform application with a consistent and responsive user interface. Firebase Authentication ensures secure user management, while Firestore provides a scalable, real-time NoSQL database for efficient data storage and retrieval. Additionally, Cloudinary integration allows for reliable and optimized handling of images, enhancing both performance and user experience.

The project also demonstrates the implementation of several important software engineering principles, including modular design, separation of concerns, and efficient state management. By organizing the application into models, services, and UI layers, the system becomes easier to maintain, extend, and debug. Furthermore, the use of validation mechanisms, structured data models, and systematic testing (both white-box and black-box) ensures the reliability and correctness of the application.

Another significant achievement of this project is its focus on user experience and usability. The application features an intuitive interface with modern design elements such as card

layouts, bottom navigation bars, search functionality, and responsive forms. These design choices make the application accessible even to users with minimal technical knowledge.

Despite being developed as an academic project, the application demonstrates real-world applicability and can serve as a foundation for a production-level system. With further enhancements such as payment gateway integration, push notifications, and advanced analytics, the platform can be transformed into a fully functional commercial solution.

In conclusion, this project not only fulfills its intended objectives but also highlights the practical application of mobile development frameworks, cloud-based backend services, and modern UI/UX design principles. It showcases the ability to conceptualize, design, implement, and evaluate a complete software system, thereby reflecting a strong understanding of both theoretical and practical aspects of software engineering.

Chapter 15: Limitations

15.1. Dummy Payment: Due to RBI regulations requiring a business license for payment gateways, I simulated a 'fake success' after checkout.

15.2. Client-side search & filter: Product search and category filtering are performed on the client side after fetching all products. For very large product catalogues, this would be inefficient and should be replaced with server-side queries (requiring composite Firestore indexes).

15.3. No push notifications: Users do not receive alerts for order confirmations, booking reminders, or event updates.

15.4. No real-time availability for cart: The cart does not check product stock in real time; it is possible to order items that go out of stock between adding to cart and checkout.

15.5. Seller analytics is recomputed on every load: For better performance, sales data could be pre-aggregated in a separate collection.

15.6. Limited report generation: Only on-screen reports; no PDF export or emailing of invoices.

15.7. No admin panel: There is no super-admin interface to manage all sellers, products, events, and users centrally.

15.8. Apple Sign-in not implemented: Only Google and email/password are supported.

Chapter 16: Future Scope

The following enhancements can be added in future versions:

16.1. Real Payment Integration: Integrate Razorpay or Stripe to accept real payments; keep dummy payment mode for testing.

16.2. Push Notifications: Use Firebase Cloud Messaging (FCM) to notify users when an order is confirmed or when an event is approaching.

16.3. Product Review & Rating: Allow customers to rate and review purchased products.

16.4. Wishlist / Save for Later: Let users bookmark products without adding to cart.

16.5. Admin Web Panel: A React or Flutter web dashboard for super-admin to approve sellers, manage platform-wide settings, and view analytics.

16.6. Server-side Pagination & Indexing: Optimise Firestore queries with composite indexes and pagination to support thousands of products/events.

16.7. Event Reminders: Local notifications 1 hour before a booked event starts.

16.8. Dark Mode: Allow users to toggle between light and dark themes.

16.9. Apple Sign-in: Add Sign in with Apple for iOS users.

16.10. Deployment to Play Store / App Store: Package the app for production release.

Chapter 17: Bibliography

17.1. Flutter Documentation. Flutter.dev. Available at: https://docs.flutter.dev/

17.2. Firebase Documentation. Firebase.google.com. Available at: https://firebase.google.com/docs

17.3. Cloudinary Flutter SDK. Pub.dev. Available at: https://pub.dev/packages/cloudinary_public

17.4. “Flutter Cookbook” by Simone Alessandria, Packt Publishing, 2021.

17.5. “Beginning Flutter: A Hands- On Guide to App Development” by Marco L. Napoli, Wiley, 2019.

17.6. “Firebase for Flutter” tutorial series by Andrea Bizzotto (Code with Andrea).

17.7. Stack Overflow – various contributions on Flutter, Firebase, and Cloudinary integration. (https://stackoverflow.com/)

17.8. Dart Language Basics (https://dart.dev/guides)

17.9. Event-Specific Features:
Calendar & Date Picker (https://pub.dev/packages/table_calendar)